Explicación del diagrama extendido de la defensa 5

Este diagrama es el extendido de la defensa 4 (`Defensa 4/Diagrama-extendido-defensa-4.drawio`) con los cambios de la defensa 5: los hilos del juego, las acciones programadas, la animación de la transición y la regla de colisiones. Todo coincide con el diagrama reducido de la defensa 5 y con sus dos explicativos (`Explicacion-diagrama-reducido-defensa-5` y `Hilos-colisiones-y-relaciones-defensa-5`).

Archivos: `Diagrama-extendido-defensa-5.drawio` y `Diagrama-extendido-defensa-5.pdf`, en esta misma carpeta.

Esta copia incluye las correcciones de la defensa 4. El ayudante de la cátedra hizo cuatro observaciones sobre la defensa 4 y el equipo las aplicó en esta carpeta. La sección 0 explica qué cambió en el extendido por esas correcciones; el resto del documento ya está actualizado con ellas.

Y las correcciones finales de colisiones. La sección Correcciones finales: colisiones explica las clases nuevas `AdministradorColisiones` y `Hitbox`, la relación borrada `#miSector` de `EntidadAnimada*`, el bloqueo con `mover(e, d, pasos)` y el cambio 4 (`# miSector` en `Entidad*`, para que cada entidad se saque sola del sector). El resto del documento también está actualizado con ellas.

Índice

- Correcciones finales: colisiones
- 0. Correcciones de la defensa 4
  - 0.1 Cambio 1: las acciones del jugador salen de `Juego` (interfaz `ControlJugador`)
  - 0.2 Cambio 2: `Sector` con una sola colección y un solo `agregar`
  - 0.3 Cambio 3: el bloqueo no usa Visitor (gancho `alChocarConBloqueo()`)
  - 0.4 Cambio 4: las estrategias de defensa pasan de cuatro a dos (`DefensaLimitada`)
- 1. Cómo leer el diagrama
- 2. Resumen de los cambios
- 3. Clases nuevas
  - 3.1 `«interface» Runnable` (java.lang)
  - 3.2 `HiloJuego*` (abstracta, implementa `Runnable`)
  - 3.3 `HiloJugador` (hereda de `HiloJuego*`)
  - 3.4 `HiloMundo` (hereda de `HiloJuego*`)
  - 3.5 `Reloj` (hereda de `HiloJuego*`, antes `Timer`)
  - 3.6 `AccionProgramada`
  - 3.7 `AnimadorTransicion` (capa gráfica, implementa `Runnable`)
- 4. Clases que cambian
  - 4.1 `Entidad*`
  - 4.2 `EntidadAnimada*`
  - 4.3 `Link`
  - 4.4 `Enemigo*` y `Proyectil`
  - 4.5 `Sector`
  - 4.6 `Juego`
  - 4.7 `ModoJuego*` y `Contrarreloj`
- 5. Lo que no se ve en el diagrama: `synchronized` y `volatile`
- 6. Relaciones nuevas
- 7. Cómo se usan juntas: tres recorridos
  - 7.1 Una vuelta de `HiloMundo`
  - 7.2 Link choca con un Moblin
  - 7.3 Transición entre sectores
- 8. Preguntas probables
- 9. Registro de cambios

Correcciones finales: colisiones

Tres correcciones sobre las colisiones y un cuarto cambio que decidió el equipo después, aplicados en esta carpeta (`Defensa 5/Correcciones finales/`). Lo nuevo o modificado está en **amarillo** (clases) y **naranja** (líneas y relaciones) en `Diagrama-extendido-defensa-5.drawio`. El detalle completo, con el porqué de cada cambio, está en Cambios-correcciones-finales.md.

N.º	Corrección	Qué se hizo en el extendido
1	La entidad no debe conocer a su sector	Sale `# miSector` de `EntidadAnimada*` con `setSector`, `getSector`, `agregarAlSector`, `quitarDelSector` y `reemplazarEnSector`. El administrador de colisiones llega por parámetro: `actualizarEstado(c)`, `actualizarPorMundo(c)`, `mover(e, c)`, `atacar(a, c)`.
2	Colisiones a nivel píxel, sin pasar a celda, en una clase aparte	Clases nuevas `AdministradorColisiones` y `Hitbox`. `Sector` queda como contenedor; `CELDA` pasa a `GeneradorNivel`.
3	El bloqueo con `puedeMoverA` y `bloqueaPaso()`	`AdministradorColisiones.mover(e, d, pasos) : boolean` pregunta el privado `puedeMoverA` y, si da false, llama al gancho `alChocarConBloqueo()`. Si la entidad sale del sector, llama al gancho nuevo `alSalirDelSector(d)`.
4	Decisión posterior del equipo: que la rupia y otras entidades se saquen del sector sin pasar por Link	`Entidad*` gana `# miSector : Sector`, `+ setSector(s : Sector)` y `+ getSector() : Sector`; `Sector` recupera `+ quitar(e : Entidad)`. `Sector.agregar(e)` hace `e.setSector(this)` y `Entidad.eliminar()` llama `miSector.quitar(this)`. Ver Cambio 4.

Clases nuevas

`AdministradorColisiones` (la compone `Juego`; la usan `HiloJugador` y `HiloMundo`)

Miembro	Para qué
`# sector : Sector`	El sector visible. Lo cambia `Juego.activarSector(s)` con `setSector(s)`.
`+ AdministradorColisiones()`, `+ setSector(s)`, `+ getSector() : Sector`	Crear y cambiar o leer el sector visible.
`+ mover(e : EntidadAnimada, d : Direccion, pasos : int) : boolean`	Avanza a `e` si nada bloquea; si no, `e.alChocarConBloqueo()` y devuelve false. Si quedó fuera, `e.alSalirDelSector(d)`.
`+ verificarColisiones(quien : EntidadAnimada)`	A cada entidad cuyo hitbox se cruza con el de `quien`: `aceptarColision(quien)` (Visitor).
`+ golpearEn(atacante, alcance : int, danio : int)`	A cada entidad dentro de `atacante.getHitbox().adelante(d, alcance)`: `recibirGolpeDeEspada`.
`- puedeMoverA(e, d, pasos) : boolean`	`e.getHitbox().desplazada(d, pasos)` contra las entidades con `bloqueaPaso()`.
`+ estaFuera(h : Hitbox) : boolean`	`!h.estaDentroDe(Sector.ANCHO, Sector.ALTO)`.

`Hitbox` (`Entidad* ◆ Hitbox`; `Hitbox → Posicion`)

Miembro	Para qué
`# posicion : Posicion`, `# ancho : int`, `# alto : int`	El rectángulo en píxeles.
`+ Hitbox(x, y, ancho, alto)`, `setPosicion`, `getPosicion`, `getX`, `getY`, `getAncho`, `getAlto`	Crear, mover y leer.
`+ intersecta(otra : Hitbox) : boolean`	Si los rectángulos se superponen.
`+ desplazada(d, pasos) : Hitbox`	El rectángulo después de moverse (para `puedeMoverA`).

Miembro	Para qué
`+ adelante(d, alcance) : Hitbox`	La zona de un golpe de espada.
`+ estaDentroDe(ancho, alto) : boolean`	Si está dentro del sector.
`+ estaEnLineaCon(otra) : boolean`	Misma fila o columna de píxeles (la trampa). Reemplaza a `Sector.estaEnLinea`.

Clases que cambian

Clase	Sale	Entra o cambia
`Entidad*`	`# posicion`, `# ancho`, `# alto`	`# hitbox : Hitbox`, `getHitbox()`, `getJuego()`, `actualizarPorMundo(c)`; cambio 4: `# miSector : Sector`, `setSector(s)`, `getSector()`
`EntidadAnimada*`	`# miSector`, `setSector`, `getSector`, `agregarAlSector`, `quitarDelSector`, `reemplazarEnSector`	`actualizarEstado(c)`, `actualizarPorMundo(c)`, `# alSalirDelSector(d)`
`Link`, `Proyectil`	—	`# alSalirDelSector(d)`: Link pide `miJuego.cambiarDeSector(d)`; el proyectil desaparece. `Link.actualizarPorMundo(c)` sigue vacío.
`Sector`	`# CELDA`, `# ancho`, `# alto`, `reemplazar`, `verificarColisiones`, `golpearEn`, `puedeMoverA`, `celdaLibreCerca`, `direccionEntre`, `estaEnLinea`, `estaFuera`, `getAncho`, `getAlto`	`+ ANCHO = 768`, `+ ALTO = 528` (estáticos), `Sector(fila, columna)`, `actualizarEntidades(c)`, `posicionLibreAlAzar(ancho, alto)`, `getCentro()` estático; `quitar(e)` se mantiene (cambio 4: la llama `e.eliminar()`)
`Juego`	`mostrarEntidad(e)`	`# colisiones : AdministradorColisiones`, `agregarEntidad(e)`
`GeneradorNivel`	—	`- CELDA : int = 48` (estático)
`Posicion`	—	`direccionHacia(otra) : Direccion`
`HiloJugador`	—	`# colisiones`, `HiloJugador(l, c)`
`HiloMundo`	`# sector`, `setSector(s)`	`# colisiones`, `HiloMundo(c)`
Estrategias de movimiento	`mover(e)`	`mover(e, c)`
Estrategias de ataque	`atacar(atacante)`, `golpear(a, danio, alcance)`, `ubicar(p, a)`	`atacar(atacante, c)`, `golpear(a, danio, alcance, c)`, `ubicar(p, a, c)`

```java
// EntidadAnimada*: una vuelta, con el administrador recibido por parámetro
public void actualizarEstado(AdministradorColisiones c) {
    vigilarJugador();
    movimiento.mover(this, c);       // la estrategia elige d; c.mover decide si puede
    ataque.atacar(this, c);
    c.verificarColisiones(this);     // efectos (Visitor)
    ejecutarPendientes();
}

// AdministradorColisiones: bloqueo y salida del sector en píxeles
public synchronized boolean mover(EntidadAnimada e, Direccion d, int pasos) {
    if (!puedeMoverA(e, d, pasos)) { e.alChocarConBloqueo(); return false; }
    e.avanzar(d, pasos);
    if (estaFuera(e.getHitbox())) e.alSalirDelSector(d);
    return true;
}
```

Cambio 4: `# miSector` en `Entidad*`

Después de las tres correcciones el equipo decidió que **toda entidad conozca a su sector** (`Entidad* → Sector`, `#miSector 1` en el reducido), para que una `Rupia`, un `Pasto`, una `Espada` o un enemigo se saquen solos del sector sin pasar por Link.

Clase	Entra	Para qué
`Entidad*`	`# miSector : Sector`, `+ setSector(s : Sector)`, `+ getSector() : Sector`	Guardar el sector donde está; lo asigna `Sector.agregar(e)`
`Sector`	`+ quitar(e : Entidad)` `(synchronized)`	Sacar la entidad de la lista; la llama la propia entidad desde `eliminar()`

```
        // Entidad*
        public void eliminar() {
            eliminada = true;
            if (miSector != null) miSector.quitar(this);   // se saca sola, sin pasar por Link
            notificarObservers();
        }
        // Sector
        public synchronized void agregar(Entidad e) { entidades.add(e); e.setSector(this); }
        public synchronized void quitar(Entidad e)  { entidades.remove(e); }
```

Qué no cambia: las colisiones siguen en `AdministradorColisiones` (llega por parámetro), las altas siguen siendo `Juego.agregarEntidad(e)` y `retirar(e)` sigue siendo para Link al cambiar de sector. Como `actualizarEntidades` y `getEntidades` recorren una copia de la lista, `quitar` puede llamarse en medio de una vuelta y `removeIf` ya no hace falta.

---

0. Correcciones de la defensa 4

Colores en esta copia: `Diagrama-extendido-defensa-5.drawio` de esta carpeta está con el formato original, sin colores que marquen cambios. Lo que cambió por las correcciones de la defensa 4 se lista en esta sección, y los cambios de la defensa 5 (hilos, acciones programadas, transición) en las secciones 1 a 9.

Observaciones del ayudante (textuales):

- "La clase Juego tiene responsabilidades que en principio no le corresponden: usarEscudo(), atacar(), ... parecen más bien responsabilidades de la clase Link."
- "En Sector tienen: agregar(e : Entidad), agregarFija(e : Entidad), agregarAnimada(a : EntidadAnimada). ¿Por qué? Si desean mantener la distinción por algún motivo, esa no sería una manera recomendable (termina derivando en un InstanceOf)."
- "Strategy: OK. Quizás es excesiva la cantidad de subclases para las estrategias de defensa."
- "Visitor: OK. Les hago una aclaración, por las dudas, no intenten modelar con Visitor el tema de los obstáculos y el bloqueo. Visitor sirve para modelar efectos que ocurren luego de interaccionar con un objeto."

0.1 Cambio 1: las acciones del jugador salen de `Juego` (interfaz `ControlJugador`)

Clase	Sale	Entra	Por qué
`«interface» ControlJugador`	(no existía)	**Nueva** (capa lógica): `+ mover(d : Direccion)`, `+ detenerMovimiento()`, `+ atacar()`, `+ soltarAtaque()`, `+ usarEscudo()`, `+`	Son las siete acciones que el jugador hace con el teclado. Son responsabilidad de Link, no de `Juego`.

Clase	Sale	Entra	Por qué
		`soltarEscudo()`, `+ cambiarObjeto()`	
`Link`	(nada)	Subtítulo "implementa ControlJugador"	Ya tenía esos siete métodos; ahora los ofrece a través de la interfaz.
`Juego`	`mover(d)`, `detenerMovimiento()`, `atacar()`, `soltarAtaque()`, `usarEscudo()`, `soltarEscudo()`, `cambiarObjeto()`	`+ getControlJugador() : ControlJugador`	`Juego` solo reenviaba esos pedidos a `jugador`. Ahora devuelve `jugador` visto como `ControlJugador`.
`«interface» ControladorJuego`	Los mismos siete métodos	`+ getControlJugador() : ControlJugador`	La GUI pide el control una vez, en vez de pasar por el controlador en cada tecla.
`GUI`	(nada)	`# controlJugador : ControlJugador`	Lo obtiene con `controladorJuego.getControlJugador()` al iniciar la partida. `procesarTecla` llama `controlJugador.mover(d)`, `controlJugador.atacar()`, etc.

Antes: tecla, `GUI.procesarTecla`, `controladorJuego.atacar()`, `Juego.atacar()`, `jugador.atacar()`. **Ahora:** tecla, `GUI.procesarTecla`, `controlJugador.atacar()`, que como es Link ejecuta `Link.atacar()`.

Por qué una interfaz y no `getJugador()`: la GUI no conoce la clase `Link` (separación de capas) y, a través de `ControlJugador`, solo ve las siete acciones del teclado, no `recibirDanio`, `curar` ni el resto de los métodos de Link.

Hilos: no cambian. Las teclas siguen llegando por el hilo de Swing (EDT); Link guarda la intención y `HiloJugador` la procesa en su vuelta.

0.2 Cambio 2: `Sector` con una sola colección y un solo `agregar`

Clase	Sale	Entra	Por qué
`Sector`	`# entidades : Entidad[][]` (lo fijo, indexado por celda) y `# animadas : List<EntidadAnimada>` (lo que se mueve)	`# entidades : List<Entidad>`	Una sola colección para todas las entidades.
`Sector`	`+ agregarFija(e : Entidad)`, `+ agregarAnimada(a : EntidadAnimada)`	Queda solo `+ agregar(e : Entidad)`	Tener tres formas de agregar obligaba a distinguir el tipo de la entidad, lo que termina en un `instanceof`. `agregar` hace `entidades.add(e)` y, si hace falta, `e.setSector(this)`.
`Entidad*`	`+ ubicarseEn(s : Sector)`	`+ actualizarPorMundo()` (gancho vacío)	Antes `agregar(e)` hacía doble despacho con `e.ubicarseEn(this)`, que respondía `agregarFija(this)` o `agregarAnimada(this)` según la entidad; con una sola lista no hace falta. El gancho reemplaza la distinción fijo/animado por polimorfismo.
`EntidadAnimada*`	`+ ubicarseEn(s : Sector)`	Redefine `+ actualizarPorMundo()`	Mueve, ataca, ejecuta pendientes y verifica colisiones (llama a `actualizarEstado()`).

La diferencia entre fijo y animado ahora es polimorfismo:

- `actualizarEntidades()` recorre todas las entidades y llama `e.actualizarPorMundo()`. Las fijas no hacen nada (gancho vacío de `Entidad*`), `EntidadAnimada*` lo redefine y `Link` lo redefine vacío porque lo maneja `HiloJugador`.
- `puedeMoverA(e, d, pasos)` recorre la lista y devuelve `false` si alguna `otra != e` cumple `otra.bloqueaPaso() && otra.intersecta(...)` con el rectángulo de la posición siguiente. Las animadas devuelven `bloqueaPaso()` en `false`, así que el comportamiento no cambia: los enemigos se siguen superponiendo entre sí.
- `verificarColisiones(quien)` recorre la lista única, salteando a `quien`.
- `# CELDA : int = 48` se queda: lo usan el parseo del archivo de nivel (fila y columna a píxeles), `celdaLibreCerca` y `posicionLibreAlAzar`. Las posiciones siguen en píxeles; lo que desaparece es guardar las entidades en una matriz indexada por celda.
- **Costo:** se pierde el acceso directo por celda. Un sector tiene 16 × 11 celdas y pocas decenas de entidades, así que recorrer la lista no es un problema.
- `synchronized` sigue en los mismos métodos de `Sector` (sección 5); ahora protegen una sola lista.

> **Después, en las correcciones finales:** `CELDA` pasó a `GeneradorNivel`, `celdaLibreCerca` se borró, `puedeMoverA`, `verificarColisiones` y `golpearEn` pasaron a `AdministradorColisiones` y `actualizarPorMundo()` recibe el administrador (`actualizarPorMundo(c)`).

0.3 Cambio 3: el bloqueo no usa Visitor (gancho `alChocarConBloqueo()`)

Clase	Sale	Entra	Por qué
`EntidadAnimada*`	(nada)	`# alChocarConBloqueo()` (gancho vacío)	Lo llama la estrategia de movimiento cuando `puedeMoverA` da `false`.
`TrampaDeCuchillas`	`+ colisionarCon(o : Obstaculo)`	`# alChocarConBloqueo()`: llama `volver()`	Al chocar con un bloqueo vuelve a su lugar.
`Proyectil`	`+ colisionarCon(o : Obstaculo)`	`# alChocarConBloqueo()`: llama `desaparecer()`	Al chocar con un bloqueo desaparece.

Ahora las dos clases heredan el `colisionarCon(o : Obstaculo)` vacío de `EntidadAnimada*`.

Por qué: antes la trampa y el proyectil reaccionaban al bloqueo con el Visitor (`colisionarCon(o : Obstaculo)`). Además era inconsistente: la estrategia de movimiento consulta `puedeMoverA` antes de avanzar, así que nunca llegaban a superponerse con la roca y ese `colisionarCon` no se ejecutaba. Ahora la estrategia hace:

```
        if (s.puedeMoverA(e, d, velocidad)) e.avanzar(d, velocidad);
        else e.alChocarConBloqueo();
```

El Visitor (`Colisionable` / `Colisionador`) queda solo para **efectos** que ocurren después de interactuar con algo: Link con pasto (`ralentizar`), caída (`caerAlVacio`), suelo quebradizo (`romperSuelo`), power-ups, objetos, enemigos y proyectiles. El bloqueo se resuelve solo con `puedeMoverA` y `bloqueaPaso()`.

0.4 Cambio 4: las estrategias de defensa pasan de cuatro a dos (`DefensaLimitada`)

Antes había cuatro clases que realizaban `ComportamientoDefensa`:

- `DefensaVulnerable`: pasaba todo el daño.
- `DefensaInvulnerable`: no pasaba nada (devolvía `false`).
- `DefensaReducida`: dejaba pasar como máximo ½ corazón por golpe durante 3 golpes y después dejaba pasar el daño completo (efecto de la bellota).
- `DefensaPorEscudo(soloDeFrente)`: bloquea según la dirección y lo que no cubre lo pasa a `#sinEscudo`.

Ahora quedan dos. `DefensaVulnerable`, `DefensaInvulnerable` y `DefensaReducida` se borraron y en su lugar está la clase nueva `DefensaLimitada`. `DefensaPorEscudo` sigue igual.

Clase	Sale	Entra	Por qué
`DefensaVulnerable`, `DefensaInvulnerable`, `DefensaReducida`	Las tres clases	(nada)	Solo diferían en cuánto daño dejaban pasar.
`DefensaLimitada` (nueva, realiza `ComportamientoDefensa`)	(nada)	`# danioMaximo : int`, `# golpesRestantes : int`, `+ SIN_LIMITE : int = Integer.MAX_VALUE` (estática, subrayada), `+ SIEMPRE : int = -1` (estática, subrayada), `+ DefensaLimitada(danioMaximo : int, golpes : int)`, `+ recibirDanio(e : EntidadAnimada, danio : int, origen : Direccion) : boolean`, `+ getGolpesRestantes() : int`	Una sola clase con el tope de daño y la cantidad de golpes como parámetros.
`DefensaPorEscudo`	(nada)	(nada)	Mismo comportamiento; `# sinEscudo : ComportamientoDefensa` ahora se inicializa con `new DefensaLimitada(DefensaLimitada.SIN_LIMITE, DefensaLimitada.SIEMPRE)`.

```
        public class DefensaLimitada implements ComportamientoDefensa {
            public static final int SIN_LIMITE = Integer.MAX_VALUE;
            public static final int SIEMPRE = -1;
            protected int danioMaximo;
            protected int golpesRestantes;
            public DefensaLimitada(int danioMaximo, int golpes) { this.danioMaximo = danioMaximo;
this.golpesRestantes = golpes; }
            public boolean recibirDanio(EntidadAnimada e, int danio, Direccion origen) {
                int real = Math.min(danio, danioMaximo);
                if (real == 0) return false;                // invulnerable: no le hace nada
                e.aplicarDanio(real);
                if (golpesRestantes != SIEMPRE && --golpesRestantes == 0) {   // fin de la bellota:
                    danioMaximo = SIN_LIMITE; golpesRestantes = SIEMPRE;          // queda sin límite
                }
                return true;
            }
            public int getGolpesRestantes() { return golpesRestantes; }
        }
```

Equivalencias entre las clases viejas y la nueva:

Antes	Ahora
`new DefensaVulnerable()`	`new DefensaLimitada(DefensaLimitada.SIN_LIMITE, DefensaLimitada.SIEMPRE)`
`new DefensaInvulnerable()`	`new DefensaLimitada(0, DefensaLimitada.SIEMPRE)`
`new DefensaReducida()` (bellota)	`new DefensaLimitada(1, 3)` (1 = ½ corazón: el daño a Link se mide en medios corazones)

Quién usa cada defensa:

Entidad	Defensa
Moblin, Octorok, Erizo, Link normal	`DefensaLimitada` sin límite
Trampa, proyectiles	`DefensaLimitada` con tope 0, siempre
Wizzrobe, MasterStalfos, StoneHinox	Alternan entre `DefensaLimitada` sin límite y con tope 0 (con `programar`)
Link 1 s tras un golpe	`DefensaLimitada` con tope 0, y vuelve a su defensa base
Link después de la bellota	`DefensaLimitada(1, 3)`; al gastar los 3 golpes queda sin límite
MoblinConEspadaYEscudo	`DefensaPorEscudo(true)` con `#sinEscudo` sin límite
Link con escudo	`DefensaPorEscudo(false)`

Por qué: el ayudante observó que quizás era excesiva la cantidad de subclases de las estrategias de defensa. Las tres defensas viejas solo diferían en cuánto daño dejaban pasar. Eso es un dato, no un comportamiento, así que pasó a ser un parámetro de `DefensaLimitada`. Queda una clase por comportamiento distinto: limitar el daño o bloquear con escudo según la dirección. Sigue siendo Strategy: la entidad cambia de defensa con `setDefensa(...)` sin `if` sobre su tipo.

---

1. Cómo leer el diagrama

- **Sin colores de cambios:** en esta copia el diagrama está con el formato original. Lo que cambió por las correcciones de la defensa 4 (`ControlJugador`, `Juego`, `ControladorJuego`, `GUI`, `Link`, `Sector`, `Entidad*`, `EntidadAnimada*`, `Proyectil`, `TrampaDeCuchillas` y la clase nueva `DefensaLimitada`) está explicado en la sección 0, y los cambios de la defensa 5 en las secciones 2 a 7.
- **Lo que se sacó** no aparece: está listado en la sección 0 y en la sección 4, clase por clase.
- **Nombre en cursiva:** clase abstracta (`HiloJuego*`). Un método en cursiva es abstracto (`# ciclo()` en `HiloJuego*`).
- **Subtítulo gris:** igual que en el resto del extendido, indica de quién hereda o qué interfaz implementa ("hereda de HiloJuego", "implementa Runnable").
- **Visibilidad:** `+` público, `#` protegido, `-` privado.

El extendido, como en la defensa 4, muestra las clases con todos sus miembros. Las flechas están en el reducido; acá cada relación aparece como un atributo (por ejemplo, `# hiloMundo : HiloMundo` en `Juego`).

---

2. Resumen de los cambios

Clase	Qué pasa	Motivo
`Runnable`	**Nueva** (interfaz de Java)	La implementan `HiloJuego*` y `AnimadorTransicion`.
`HiloJuego*`	**Nueva**, abstracta	Base común de los hilos de la lógica (Template Method).
`HiloJugador`	**Nueva**	Mueve a Link cada 30 ms y resuelve sus choques.
`HiloMundo`	**Nueva**	Mueve a los enemigos y proyectiles del sector visible cada 30 ms.
`Reloj`	**Reemplaza a `Timer`**	Mismo trabajo (cuenta segundos), ahora hereda de `HiloJuego*`.
`AccionProgramada`	**Nueva**	Reemplaza los hilos que creaba `programar(ms, accion)`.
`AnimadorTransicion`	**Nueva** (capa gráfica)	Reemplaza el `Thread` anónimo de `GUI.transicion(d)`.
`Entidad*`	Cambia	Salen `iniciar()` y `detener()`. Por las correcciones de la defensa 4, sale `ubicarseEn(s)` y entra el gancho vacío `actualizarPorMundo()`.

Clase	Qué pasa	Motivo
`EntidadAnimada*`	Cambia	Ya no tiene hilo propio: entra `actualizarEstado()` y la lista de acciones programadas. Por las correcciones de la defensa 4, sale `ubicarseEn(s)` y entra `alChocarConBloqueo()`.
`Link`	Cambia	Redefine vacío `actualizarPorMundo()`. Por las correcciones de la defensa 4, implementa `ControlJugador`.
`Enemigo*`, `Proyectil`	Cambian	Sale `colisionarCon(l : Link)`: queda el gancho vacío heredado. Por las correcciones de la defensa 4, `Proyectil` (y también `TrampaDeCuchillas`) cambia `colisionarCon(o : Obstaculo)` por `alChocarConBloqueo()`.
`Sector`	Cambia	Salen `iniciarEntidades()` y `detenerEntidades()`; entra `actualizarEntidades()`. Por las correcciones de la defensa 4, guarda una sola `List<Entidad>` y tiene un solo `agregar(e)`.
`AdministradorColisiones`, `Hitbox`	Nuevas (correcciones finales)	Colisiones, bloqueo, golpes y salida del sector en píxeles. Ver Correcciones finales: colisiones, que también lista lo que cambia en `Entidad*`, `EntidadAnimada*`, `Sector`, `Juego`, los hilos y las estrategias.
`Juego`	Cambia	Es dueño de los tres hilos (`reloj`, `hiloJugador`, `hiloMundo`). Por las correcciones de la defensa 4, pierde las siete acciones del jugador y gana `getControlJugador()`.
`ControlJugador`	**Nueva** (corrección de la defensa 4)	Interfaz con las siete acciones del teclado; la implementa `Link`. Ver sección 0.
`ModoJuego*`, `Contrarreloj`	Cambian	Gancho `alIniciarNivel()`; `Contrarreloj` descuenta `tiempoRestante`.

Lo que no cambia: la parte gráfica (salvo la nueva `AnimadorTransicion`), las fábricas, los comportamientos (Strategy), los power-ups, los obstáculos, los objetos y el doble despacho de colisiones (`Colisionable` / `Colisionador`), que desde las correcciones de la defensa 4 se usa solo para efectos y no para el bloqueo. `GUI` mantiene los mismos métodos: por los hilos solo cambia lo que hace por dentro `transicion(d)`, y por las correcciones de la defensa 4 gana el atributo `# controlJugador : ControlJugador`.

---

3. Clases nuevas

3.1 `«interface» Runnable` (java.lang)

Miembro	Para qué
`+ run()`	Lo que ejecuta el hilo. Un objeto `Runnable` se le pasa a un `Thread` (`new Thread(this).start()`) y ese hilo llama a `run()`.

Aparece en el diagrama por la misma razón que `JPanel` o `JLabel`: muestra de qué clase de Java dependen las clases propias. La implementan `HiloJuego*` (y por herencia sus tres subclases) y `AnimadorTransicion`.

3.2 `HiloJuego*` (abstracta, implementa `Runnable`)

Base común de `HiloJugador`, `HiloMundo` y `Reloj`. Sabe arrancar, pausar, reanudar y detener un hilo; cada subclase dice solo qué hacer en una vuelta. Es el patrón **Template Method**: `run()` es el esquema fijo y `ciclo()` el paso que cambia.

Miembro	Tipo	Para qué
`# hilo : Thread`	atributo	El hilo de Java que ejecuta `run()`. Se crea en `iniciar()`.

Miembro	Tipo	Para qué
`# activo : boolean`	atributo	Mientras sea `true`, el hilo sigue. Es `volatile` (sección 5).
`# pausado : boolean`	atributo	Si es `true`, el hilo da vueltas sin hacer nada (transición). `volatile`.
`# periodo : long`	atributo	Milisegundos entre vueltas: 30 para los hilos de juego, 1000 para el reloj.
`# HiloJuego(periodo : long)`	constructor	Lo llaman las subclases con `super(30)` o `super(1000)`.
`+ iniciar()`	comando	`activo = true; hilo = new Thread(this); hilo.start();`
`+ detener()`	comando	`activo = false`: el hilo termina al final de la vuelta.
`+ pausar()` / `+ reanudar()`	comandos	`pausado = true` / `false`. Los usa `Juego` en la transición.
`+ run()`	comando	Fijo: `while (activo) { if (!pausado) ciclo(); Thread.sleep(periodo); }`.
`# ciclo()`	comando abstracto	Una vuelta. La redefine cada subclase.
`+ estaActivo() : boolean`	consulta	Si el hilo está corriendo (venía de `Timer`).
`+ estaPausado() : boolean`	consulta	Si está en pausa.

```
public abstract class HiloJuego implements Runnable {
    protected Thread hilo;
    protected volatile boolean activo, pausado;
    protected long periodo;
    protected HiloJuego(long periodo) { this.periodo = periodo; }
    public void iniciar()  { activo = true; hilo = new Thread(this); hilo.start(); }
    public void detener()  { activo = false; }
    public void pausar()   { pausado = true; }
    public void reanudar() { pausado = false; }
    public final void run() {
        while (activo) {
            if (!pausado) ciclo();
            try { Thread.sleep(periodo); } catch (InterruptedException e) { activo = false; }
        }
    }
    protected abstract void ciclo();
    public boolean estaActivo() { return activo; }
    public boolean estaPausado() { return pausado; }
}
```

3.3 `HiloJugador` (hereda de `HiloJuego*`)

Miembro	Tipo	Para qué
`# jugador : Link`	atributo	El Link que actualiza.
`# colisiones : AdministradorColisiones`	atributo	Correcciones finales: se lo pasa a Link en cada vuelta.
`+ HiloJugador(l : Link, c : AdministradorColisiones)`	constructor	`super(30); jugador = l; colisiones = c;`
`# ciclo()`	comando	`jugador.actualizarEstado(colisiones);`: mueve a Link, ataca, revisa choques y ejecuta sus acciones vencidas.

Por qué existe: Link responde al teclado sin esperar a que se muevan los enemigos, y todos los choques en los que participa Link se resuelven en este hilo (regla de colisiones).

3.4 `HiloMundo` (hereda de `HiloJuego*`)

Miembro	Tipo	Para qué
`# colisiones : AdministradorColisiones`	atributo	Correcciones finales: reemplaza a `# sector`. El sector visible lo guarda el administrador.
`+ HiloMundo(c : AdministradorColisiones)`	constructor	`super(30); colisiones = c;`
`# ciclo()`	comando	`colisiones.getSector().actualizarEntidades(colisiones);`

`setSector` salió de `HiloMundo`: `Juego.activarSector(s)` hace `colisiones.setSector(s)` y los dos hilos ven el sector nuevo a la vez.

Por qué uno solo y no un hilo por enemigo: un hilo que recorre a todos en orden evita decenas de hilos tocando el sector a la vez.

3.5 `Reloj` (hereda de `HiloJuego*`, antes `Timer`)

Miembro	Tipo	Para qué
`# segundos : int`	atributo	Segundos de la partida.
`# juego : Juego`	atributo	A quién avisar cada segundo.
`+ Reloj(j : Juego)`	constructor	`super(1000); juego = j;`
`+ reiniciar()`	comando	`segundos = 0`.
`# ciclo()`	comando	`segundos++; juego.tic(segundos);`
`+ getSegundos() : int`	consulta	Segundos actuales.

Qué se fue de `Timer`: `# activo`, `iniciar()`, `detener()`, `run()` y `estaActivo()`. No desaparecieron: ahora los hereda de `HiloJuego*`. **Por qué el nombre nuevo:** el enunciado prohíbe usar timers para los movimientos; la clase nunca fue `javax.swing.Timer`, pero el nombre invitaba a la confusión.

3.6 `AccionProgramada`

Algo que una entidad tiene que hacer dentro de un rato ("en 1000 ms, volvé a ser vulnerable"). Se guarda en la lista `pendientes` de la entidad y la ejecuta el mismo hilo que la actualiza, cuando vence. No crea hilos.

Miembro	Tipo	Para qué
`# instante : long`	atributo	Cuándo vence: `System.currentTimeMillis() + ms`.
`# accion : Runnable`	atributo	Qué hacer. Es un `Runnable` guardado como dato, por ejemplo `() -> restaurarDefensa()`.
`+ AccionProgramada(instante : long, accion : Runnable)`	constructor	La crea `EntidadAnimada.programar(ms, accion)`.
`+ ejecutar()`	comando	`accion.run()`, en el hilo que la llama (no crea un `Thread`).
`+ estaVencida(ahora : long) : boolean`	consulta	`ahora >= instante`.

Tiene un `Runnable` pero no es un hilo: no implementa `Runnable` ni hereda de `Thread`. Por eso no lleva el subtítulo "implementa Runnable".

Dónde se usa: invulnerabilidad de Link (1000 ms) y de MasterStalfos (800 ms), suelo quebradizo (500 ms), vuelta de la trampa (600 ms), fin de la embestida del Moblin con escudo y la cadena de ataques de StoneHinox (1500 + 1000 ms).

3.7 `AnimadorTransicion` (capa gráfica, implementa `Runnable`)

Miembro	Tipo	Para qué
`# panel : PanelPantallaNivel`	atributo	El panel que se desplaza.
`# direccion : Direccion`	atributo	Hacia dónde.
`# controlador : ControladorJuego`	atributo	A quién avisar al terminar.
`+ AnimadorTransicion(p : PanelPantallaNivel, d : Direccion, c : ControladorJuego)`	constructor	La crea `GUI.transicion(d)`.
`+ run()`	comando	`panel.desplazar(direccion)` paso a paso y, al terminar, `controlador.transicionTerminada()`.

Respeta las capas: conoce un panel de la vista y la **interfaz** `ControladorJuego`, nunca `Juego` directamente.

```
        // GUI
        public void transicion(Direccion d) {
            new Thread(new AnimadorTransicion(panelNivel, d, controladorJuego)).start();
        }
```

---

4. Clases que cambian

En cada tabla, **Sale** es lo que estaba en la defensa 4 y ya no está, y **Entra** es lo nuevo.

4.1 `Entidad*`

Sale	Entra	Por qué
`+ iniciar()`, `+ detener()`	—	Ninguna entidad tiene hilo propio. Las animadas las mueven `HiloJugador` y `HiloMundo`; las fijas nunca tuvieron que hacer nada.

4.2 `EntidadAnimada*`

Sale	Entra	Por qué
`# activa : boolean`	`# pendientes : List<AccionProgramada>`	`activa` decía si el hilo propio corría; ya no hay hilo propio. La lista guarda las acciones programadas (composición, `0..*`).
`+ iniciar()`, `+ detener()`, `+ run()`	`+ actualizarEstado(c)`	En vez de un `while` con `sleep`, la entidad da **una** vuelta cuando se lo pide su hilo.
—	`+ actualizarPorMundo(c)`	Gancho que llama `HiloMundo` (vía `Sector.actualizarEntidades(c)`). Desde las correcciones de la defensa 4 se declara vacío en `Entidad*` (las fijas no hacen nada) y `EntidadAnimada*` lo redefine para llamar a `actualizarEstado()`; `Link` lo redefine vacío.
`+ ubicarseEn(s : Sector)`	`# alChocarConBloqueo()`	Correcciones de la defensa 4: `ubicarseEn` ya no hace falta porque el sector tiene una sola lista; `alChocarConBloqueo()` es el gancho vacío que llama la estrategia de movimiento cuando `puedeMoverA` da `false` (sección 0.3).
—	`# vigilarJugador()`	Gancho vacío del principio de la vuelta. Lo redefinen `Enemigo*` (y la trampa y el Moblin con escudo, que se abalanzan).
—	`- ejecutarPendientes()`	Al final de la vuelta, ejecuta las acciones vencidas.

Sale	Entra	Por qué
—	`- sacarVencidas(ahora : long) : List<AccionProgramada>`	Saca las vencidas de la lista (con la llave tomada) para ejecutarlas después, fuera de la llave.
`+ estaActiva() : boolean`	—	Preguntaba si el hilo propio corría. Para saber si una entidad sigue en juego está `estaEliminada()`.

Lo que se mantiene: `# programar(ms : long, accion : Runnable)` con la misma firma, así las subclases que lo usan no cambian; ahora anota una `AccionProgramada` en vez de crear un hilo. `recibirDanio` y `aplicarDanio` pasan a ser `synchronized` (sección 5).

```
        public void actualizarEstado(AdministradorColisiones c) {   // una vuelta: la llama
HiloJugador (Link) o HiloMundo (el resto)
            vigilarJugador();
            movimiento.mover(this, c);
            ataque.atacar(this, c);
            c.verificarColisiones(this);
            ejecutarPendientes();
        }
        public void actualizarPorMundo(AdministradorColisiones c) { actualizarEstado(c); }   // Link
lo redefine vacío
        protected void alChocarConBloqueo() { }                    // gancho: TrampaDeCuchillas
vuelve, Proyectil desaparece
        protected void alSalirDelSector(Direccion d) { }           // gancho (correcciones finales):
Link cambia de sector, Proyectil desaparece

        protected synchronized void programar(long ms, Runnable accion) {
            pendientes.add(new AccionProgramada(System.currentTimeMillis() + ms, accion));
        }
        private void ejecutarPendientes() {
            for (AccionProgramada a : sacarVencidas(System.currentTimeMillis())) a.ejecutar();
        }
        private synchronized List<AccionProgramada> sacarVencidas(long ahora) {
            List<AccionProgramada> vencidas = new ArrayList<>();
            for (Iterator<AccionProgramada> it = pendientes.iterator(); it.hasNext(); ) {
                AccionProgramada a = it.next();
                if (a.estaVencida(ahora)) { vencidas.add(a); it.remove(); }
            }
            return vencidas;
        }
```

4.3 `Link`

Sale	Entra	Por qué
—	`+ actualizarPorMundo(c)` (vacío)	Link está en la lista `entidades` del sector, pero a Link lo mueve `HiloJugador`. Si `HiloMundo` también llamara su `actualizarEstado(c)`, Link se movería dos veces por vuelta. El gancho vacío lo evita sin preguntar el tipo.

Por las correcciones de la defensa 4, `Link` además implementa `ControlJugador` (sección 0.1): la GUI le manda las siete acciones del teclado a través de esa interfaz, sin pasar por `Juego`.

Además, sin cambiar firmas: las intenciones que escribe el teclado (`direccion`, `atacando`, `soltoAtaque`, `escudoActivo`) son `volatile`, y `cargarAtaque()` solo empieza a cargar si `!atacando` (Swing repite `keyPressed` mientras la tecla está apretada).

4.4 `Enemigo*` y `Proyectil`

Sale	Entra	Por qué
`+ colisionarCon(l : Link)`	—	Regla de colisiones: todo choque con Link lo resuelve `HiloJugador`, en `Link.colisionarCon(Enemigo)` o `Link.colisionarCon(Proyectil)`. Del lado del enemigo queda el `colisionarCon(l : Link)` vacío heredado de `EntidadAnimada*`. Si los dos lados aplicaran el efecto, se aplicaría desde dos hilos y dependería de cuál llega primero.
`+ colisionarCon(o : Obstaculo)` (solo `Proyectil`)	`# alChocarConBloqueo()`	Correcciones de la defensa 4: el proyectil ya no reacciona al bloqueo con el Visitor; cuando `puedeMoverA` da `false`, `AdministradorColisiones.mover` (correcciones finales; antes, la estrategia de movimiento) llama `alChocarConBloqueo()` y el proyectil hace `desaparecer()`. `TrampaDeCuchillas` hace el mismo cambio y en el gancho llama `volver()` (sección 0.3).

4.5 `Sector`

Sale	Entra	Por qué
`+ iniciarEntidades()`, `+ detenerEntidades()`	`+ actualizarEntidades(c)`	El sector ya no arranca hilos: `HiloMundo` le pide una vuelta cada 30 ms.
`# entidades : Entidad[][]`, `# animadas : List<EntidadAnimada>`, `+ agregarFija(e)`, `+ agregarAnimada(a)`	`# entidades : List<Entidad>` y un solo `+ agregar(e : Entidad)`	Correcciones de la defensa 4: una sola colección, sin distinguir el tipo al agregar (sección 0.2).

```
        public synchronized void actualizarEntidades(AdministradorColisiones c) {
            for (Entidad e : new ArrayList<>(entidades))   // copia: la lista puede cambiar en medio
                if (!e.estaEliminada()) e.actualizarPorMundo(c);   // las fijas no hacen nada; Link tampoco
        }
        public synchronized void quitar(Entidad e) { entidades.remove(e); }   // cambio 4: la llama e.eliminar()
```

4.6 `Juego`

Sale	Entra	Por qué
`# timer : Timer`	`# reloj : Reloj`	Cambio de nombre de la clase.
—	`# hiloJugador : HiloJugador`	Se crea en `iniciar` junto con Link.
—	`# hiloMundo : HiloMundo`	Recorre el sector activo.
`mover(d)`, `detenerMovimiento()`, `atacar()`, `soltarAtaque()`, `usarEscudo()`, `soltarEscudo()`, `cambiarObjeto()`	`+ getControlJugador() : ControlJugador`	Correcciones de la defensa 4: esas acciones son de Link. `getControlJugador()` devuelve `jugador` como `ControlJugador` (sección 0.1). `ControladorJuego` hace el mismo cambio.

Sin cambiar firmas, cambian por dentro:

Método	Qué hace con los hilos
`iniciar(modo, dominio)`	Crea `hiloJugador = new HiloJugador(jugador, colisiones)` y `hiloMundo = new HiloMundo(colisiones)`, y arranca los tres hilos.
`activarSector(s)` (privado)	Además de armar las celdas, hace `colisiones.setSector(s)` (correcciones finales; antes `hiloMundo.setSector(s)`).

Método	Qué hace con los hilos
`cambiarDeSector(d)`	`hiloJugador.pausar(); hiloMundo.pausar();` antes de pedir la transición. El reloj sigue.
`transicionTerminada()`	Activa el sector nuevo y hace `reanudar()` en los dos hilos.
`terminarPartida()`	`detener()` en los tres hilos.
`cargarNivel()` (privado)	Llama `modoJuego.alIniciarNivel()`.

4.7 `ModoJuego*` y `Contrarreloj`

Clase	Sale	Entra	Por qué
`ModoJuego*`	—	`+ alIniciarNivel()`	Gancho vacío que `Juego` llama al cargar cada nivel.
`Contrarreloj`	—	`# tiempoRestante : int`	El tiempo que queda; empieza en `tiempoLimite`.
`Contrarreloj`	—	`+ alIniciarNivel()`	`tiempoRestante = tiempoLimite`.
`Contrarreloj`	—	`+ getTiempoRestante() : int`	Para mostrar la cuenta regresiva.

`alPasarTiempo(j, segundos)` mantiene la firma; ahora hace `tiempoRestante--` y, si llega a 0, `j.terminarPartida()`. Así el reloj "descuenta un segundo al modo de juego vigente", como dice el diagrama de secuencia 3.

---

5. Lo que no se ve en el diagrama: `synchronized` y `volatile`

UML no tiene una marca estándar para `synchronized` ni `volatile`, así que no se agregan al diagrama. Son parte del diseño y conviene saberlos para la defensa.

Dónde	Qué	Por qué
`Sector`: `agregar`, `retirar`, `getEntidades`, `actualizarEntidades`; `AdministradorColisiones`: `mover`, `verificarColisiones`	`synchronized`	`HiloJugador`, `HiloMundo` y `Juego` tocan la misma lista `entidades` (desde las correcciones de la defensa 4 hay una sola).
`EntidadAnimada*`: `recibirDanio`, `aplicarDanio`	`synchronized`	Una espada (`HiloJugador`) y una roca (`HiloMundo`) pueden dañar al mismo enemigo a la vez; sin la llave se pierde un golpe.
`EntidadAnimada*`: `programar`, `sacarVencidas`	`synchronized`	Otro hilo puede anotar en la lista de una entidad (MasterStalfos golpea a Link y anota en la lista de Link).
`Juego`: `powerUpRecogido`, `enemigoDerrotado`, `jugadorDaniado`, `tic`, `terminarPartida`	`synchronized`	Puntaje, vidas y tiempo los tocan varios hilos.
`HiloJuego*`: `activo`, `pausado`	`volatile`	Los cambia un hilo y los lee otro.
`Link`: `direccion`, `atacando`, `soltoAtaque`, `escudoActivo`	`volatile`	Los escribe el teclado (EDT de Swing) y los lee `HiloJugador`.
`CeldaGrafica.actualizar()`	`SwingUtilities.invokeLater`	Solo el hilo de Swing puede tocar componentes gráficos.

---

6. Relaciones nuevas

En el extendido las relaciones se ven como atributos. Estas son las nuevas (las mismas que en el reducido):

Relación	Tipo	Atributo que la representa
`HiloJuego*` implementa `Runnable`	Realización	subtítulo "implementa Runnable"
`HiloJugador`, `HiloMundo`, `Reloj` heredan de `HiloJuego*`	Herencia	subtítulo "hereda de HiloJuego"
`Juego ◆ Reloj`	Composición (ya estaba)	`Juego.# reloj : Reloj`
`Juego ◆ HiloJugador`	Composición	`Juego.# hiloJugador : HiloJugador`
`Juego ◆ HiloMundo`	Composición	`Juego.# hiloMundo : HiloMundo`
`Reloj → Juego`	Asociación (ya estaba)	`Reloj.# juego : Juego`
`HiloJugador → Link`	Asociación	`HiloJugador.# jugador : Link`
`HiloMundo → AdministradorColisiones` (antes `HiloMundo → Sector`)	Asociación (correcciones finales)	`HiloMundo.# colisiones : AdministradorColisiones`
`HiloJugador → AdministradorColisiones`	Asociación (correcciones finales)	`HiloJugador.# colisiones : AdministradorColisiones`
`Juego ◆ AdministradorColisiones`	Composición (correcciones finales)	`Juego.# colisiones : AdministradorColisiones`
`AdministradorColisiones → Sector`	Asociación (correcciones finales)	`AdministradorColisiones.# sector : Sector`
`Entidad* ◆ Hitbox` y `Hitbox → Posicion`	Composición y asociación (correcciones finales)	`Entidad.# hitbox : Hitbox`, `Hitbox.# posicion : Posicion`
`EntidadAnimada* ◆ AccionProgramada (0..*)`	Composición	`EntidadAnimada.# pendientes : List<AccionProgramada>`
`AnimadorTransicion` implementa `Runnable`	Realización	subtítulo "implementa Runnable"
`AnimadorTransicion → PanelPantallaNivel`	Asociación	`AnimadorTransicion.# panel : PanelPantallaNivel`
`AnimadorTransicion → ControladorJuego`	Asociación	`AnimadorTransicion.# controlador : ControladorJuego`
`Link` implementa `ControlJugador`	Realización (corrección de la defensa 4)	subtítulo "implementa ControlJugador"
`GUI → ControlJugador`	Asociación (corrección de la defensa 4)	`GUI.# controlJugador : ControlJugador`

Por las correcciones de la defensa 4 también se fue `Sector → EntidadAnimada*` (el atributo `animadas`); queda `Sector → Entidad*` con `# entidades : List<Entidad>`. Por las correcciones finales se fue `EntidadAnimada* → Sector` (`# miSector`) y `Entidad* → Posicion` pasó a ser `Hitbox → Posicion`. Con el cambio 4 apareció `Entidad* → Sector` (`# miSector` en `Entidad*`), que se usa solo para que la entidad se saque con `eliminar()`.

Composición con los hilos: se crean con el juego y no tienen sentido sin él. **Composición con `AccionProgramada`:** cada acción la crea una entidad, es solo suya y desaparece con ella. **Asociación `AdministradorColisiones → Sector`:** el sector no es del administrador; solo lo recorre, y cambia en cada transición.

---

7. Cómo se usan juntas: tres recorridos

7.1 Una vuelta de `HiloMundo`

`HiloJuego.run()` → `HiloMundo.ciclo()` → `colisiones.getSector().actualizarEntidades(colisiones)` → para cada entidad de la lista, `actualizarPorMundo(c)` (en las fijas y en Link, nada) → `actualizarEstado(c)` → `vigilarJugador()`, `mover(this, c)` (→ `c.mover`), `atacar(this, c)`, `c.verificarColisiones(this)`,

`ejecutarPendientes()`. Si en la vuelta una entidad hace `eliminar()`, ella misma se saca con `miSector.quitar(this)` (cambio 4); el recorrido sigue sobre la copia de la lista.

7.2 Link choca con un Moblin

1. `HiloMundo`, vuelta del Moblin: `c.verificarColisiones(moblin)` → `link.aceptarColision(moblin)` → `moblin.colisionarCon(link)` → gancho vacío heredado, no pasa nada.
2. `HiloJugador`, vuelta de Link: `c.verificarColisiones(link)` → `moblin.aceptarColision(link)` → `link.colisionarCon(moblin)` → `recibirDanio(...)` (`synchronized`): pierde un corazón.
3. `alSerGolpeado()` → `setDefensa(new DefensaLimitada(0, DefensaLimitada.SIEMPRE))` y `programar(1000, () -> restaurarDefensa())`: se anota una `AccionProgramada` en `pendientes` de Link.
4. 1000 ms después, en una vuelta de `HiloJugador`, `sacarVencidas` la saca y `ejecutar()` vuelve a la defensa normal.

7.3 Transición entre sectores

1. `HiloJugador`: `c.mover` ve que Link salió del sector → `link.alSalirDelSector(d)` → `Juego.cambiarDeSector(d)` → `hiloJugador.pausar()`, `hiloMundo.pausar()`.
2. `Juego` → `controladorGrafico.transicion(d)` → `GUI` crea `new Thread(new AnimadorTransicion(panelNivel, d, controladorJuego)).start()`.
3. Hilo del animador: `panel.desplazar(d)` → `controlador.transicionTerminada()`.
4. `Juego.transicionTerminada()` → `activarSector(nuevo)` → `colisiones.setSector(nuevo)` → `reanudar()` en los dos hilos.

---

8. Preguntas probables

Pregunta	Respuesta
¿Por qué ya no está `Timer`?	Pasó a llamarse `Reloj` y hereda de `HiloJuego*`. Hace lo mismo: cuenta segundos y avisa `tic` a `Juego`.
¿Dónde quedaron `iniciar()` y `detener()` de las entidades?	Ya no hacen falta: ninguna entidad tiene hilo. Arrancar y detener lo hacen los hilos, desde `Juego`.
¿Quién llama a `actualizarEstado(c)`?	`HiloJugador` para Link; `HiloMundo` (vía `Sector.actualizarEntidades(c)` y `actualizarPorMundo(c)`) para el resto. Los dos le pasan el mismo `AdministradorColisiones`.
¿Por qué las entidades reciben `c` por parámetro?	Corrección final: la entidad no debe conocer a su sector. Con el parámetro, la entidad usa el administrador solo durante su vuelta (dependencia) y no guarda ninguna referencia.
¿Dónde quedaron las celdas?	Solo en `GeneradorNivel.CELDA`, para leer el archivo. Las colisiones comparan rectángulos en píxeles (`Hitbox`).
¿Por qué `Link` redefine vacío `actualizarPorMundo(c)`?	Para que `HiloMundo` no lo mueva: lo mueve `HiloJugador`. Así no hace falta preguntar si la entidad es Link.
¿Por qué sacaron `colisionarCon(Link)` de `Enemigo*` y `Proyectil`?	Para que un choque con Link se resuelva una sola vez, en el hilo de Link. Del lado del enemigo queda el gancho vacío heredado.
¿`AccionProgramada` es un hilo?	No. Guarda un `Runnable` como dato y lo ejecuta con un llamado normal, dentro de la vuelta de la entidad.
¿Por qué `HiloJuego*` es abstracta?	Porque no sabe qué hacer en una vuelta: eso lo dice cada subclase en `ciclo()`.
¿Por qué `AnimadorTransicion` está en la capa gráfica?	Mueve un panel de Swing. Conoce la interfaz `ControladorJuego`, no `Juego`, así respeta las capas.
¿Dónde se ve `synchronized`?	No se ve: UML no tiene una marca estándar. Está en la sección 5.

Pregunta	Respuesta
¿Por qué `ControlJugador` y no que la GUI use `getJugador()`?	Porque la GUI no debe conocer la clase `Link` (separación de capas) y, con la interfaz, solo ve las siete acciones del teclado. Además esas acciones son responsabilidad de Link, no de `Juego`, como observó el ayudante.
¿Por qué una sola lista en `Sector`?	Con `agregarFija` y `agregarAnimada` había que distinguir el tipo de cada entidad, lo que termina en un `instanceof`. Con una sola lista la diferencia la resuelve el polimorfismo: `actualizarPorMundo(c)` y `bloqueaPaso()`.
¿Por qué el bloqueo no usa el Visitor?	El Visitor sirve para efectos que ocurren después de interactuar con algo. El bloqueo se resuelve antes de moverse, con `puedeMoverA` y `bloqueaPaso()` dentro de `AdministradorColisiones.mover`; si no se puede avanzar, el administrador llama `alChocarConBloqueo()`.
¿No son muchas las estrategias de defensa?	Ahora son dos. `DefensaVulnerable`, `DefensaInvulnerable` y `DefensaReducida` solo diferían en cuánto daño dejaban pasar, que es un dato y no un comportamiento, así que se unieron en `DefensaLimitada` con el tope de daño y los golpes como parámetros. `DefensaPorEscudo` queda aparte porque sí tiene comportamiento propio: depende de la dirección y envuelve a otra defensa (sección 0.4).

---

9. Registro de cambios

Fecha	Cambio
20/10/2026	Se crea el extendido de la defensa 5 a partir del de la defensa 4: seis clases nuevas (`Runnable`, `HiloJuego*`, `HiloJugador`, `HiloMundo`, `AccionProgramada` y `AnimadorTransicion`), `Reloj` en lugar de `Timer` y cambios en `Entidad*`, `EntidadAnimada*`, `Link`, `Enemigo*`, `Proyectil`, `Sector`, `Juego`, `ModoJuego*` y `Contrarreloj`.
20/10/2026	Correcciones de la defensa 4 (sección 0): nueva interfaz `ControlJugador` (la implementa `Link`); `Juego` y `ControladorJuego` pierden las siete acciones del jugador y ganan `getControlJugador()`; `GUI` gana `controlJugador`; `Sector` guarda una sola `List<Entidad>` y pierde `animadas`, `agregarFija` y `agregarAnimada`; `Entidad*` y `EntidadAnimada*` pierden `ubicarseEn`; `actualizarPorMundo()` se declara en `Entidad*`; nuevo gancho `alChocarConBloqueo()` que reemplaza `colisionarCon(o : Obstaculo)` en `Proyectil` y `TrampaDeCuchillas`. El amarillo y el naranja de esta copia marcan solo estas correcciones.
20/10/2026	Defensas: `DefensaVulnerable`, `DefensaInvulnerable` y `DefensaReducida` se unen en `DefensaLimitada` (corrección 4 del ayudante, sección 0.4).
20/10/2026	El extendido de esta carpeta queda con el formato original: se sacaron el amarillo y el naranja que marcaban las correcciones.
21/10/2026	Correcciones finales de colisiones (sección nueva al principio): clases `AdministradorColisiones` y `Hitbox`; sale `# miSector` y los métodos de sector de `EntidadAnimada*`; parámetro `c : AdministradorColisiones` en `actualizarEstado`, `actualizarPorMundo`, `mover`, `atacar`, `golpear` y `ubicar`; gancho `alSalirDelSector(d)`; `Sector` sin lo espacial y con `ANCHO/ALTO` estáticos; `CELDA` en `GeneradorNivel`; `Juego.agregarEntidad(e)`; `HiloMundo` y `HiloJugador` con `# colisiones`. Lo nuevo está en amarillo y naranja en el extendido de esta carpeta.
21/10/2026	Cambio 4 de las correcciones finales (decisión del equipo): `Entidad*` gana `# miSector`, `setSector(s)` y `getSector()`; `Sector` recupera `quitar(e)`. `agregar` hace `e.setSector(this)` y `eliminar()` llama `miSector.quitar(this)`, así la rupia y las demás entidades se sacan solas sin pasar por Link; `removeIf` ya no hace falta. Subsección nueva "Cambio 4" y se actualizan 4.5, 6 y 7.1.